- You can provide a lambda factory function that describes how to create an instance of a service with any lifetime you choose. You can take this approach when your services depend on other services that are accessible only when your application is running.
- Avoid calling `GetService()` or `GetRequiredService()` in your factory functions if possible. Instead, favor constructor injection; it's more performant and simpler to reason about.
- You can register multiple implementations for a service. Then you can inject `IEnumerable<T>` to get access to all the implementations at runtime.
- If you inject a single instance of a multiple-registered service, the container injects the last implementation registered.
- You can use the `TryAdd*` extension methods to ensure that an implementation is registered only if no other implementation of the service has been registered. This approach can be useful for library authors to add default services while still allowing consumers to override the registered services.
- You define the lifetime of a service during DI service registration to dictate when a DI container will reuse an existing instance of the service to fulfill service dependencies and when it will create a new one.
- A transient lifetime means that every time a service is requested, a new instance is created.